

Epic / GitHub project requirements status ▾

login and password management
of sent and received messages
sending a new message
message to another user after verifyin
er's access to a previously shared me
a message (owned or shared)
essage

take a look at the github in the project files - what's the current state on all the requirements - what needs to be done

Ran 15 commands, searched memory, viewed a file >

Now I have a very complete picture. Let me compile this into a clear status report.

Ran 3 commands >

Now I have everything I need. Here's the full picture:

EPIC Requirements — Current State

Here's exactly where things stand against each requirement from the spec:

✅ User sign-up, login and password management

Status: ~80% done

- Register, login, and logout all work end-to-end (backend + C++ client + web frontend)
 - Password is hashed with Argon2id correctly
 - **Gap:** No *password change* endpoint exists anywhere in the codebase — the auth router only has register/login/logout/me/upload-prekeys/keybundle. "Password management" in the rubric likely implies changing it. Low-effort to add but currently missing.
-

✅ Viewing a list of sent and received messages

Status: Done

- Backend has `GET /messages/inbox` and `GET /messages/sent` with pagination and cursor support
 - C++ client's `Client.cpp` exposes `fetchInbox()` and `fetchSent()`
 - Web frontend `app.js` has `fetchInbox()` with polling
 - MainWindow renders thread views with load-older support
-

✅ Creating and sending a new message

Status: Done

- Full E2E path: Double Ratchet encrypt in daemon → `POST /messages/send` → WebSocket push to recipient
 - Works in both C++ client and web frontend
 - X3DH handshake on first message, ratchet-only after
-

⚠️ Forwarding a message to another user after verifying their identity

Status: Backend done, client not wired up

- Backend: `POST /messages/{message_id}/forward` is fully implemented, with blockchain recording
 - `ForwardMessageRequest` / `ForwardMessageResponse` schemas exist
 - **Gap:** Neither the C++ client (`Client.hpp` has no `forwardMessage()` method) nor the web frontend `app.js` (zero occurrences of "forward") expose this to the user. The feature is invisible at demo time.
-

⚠️ Revoking a user's access to a previously shared message

Status: Backend done, client not wired up

- Backend: `POST /api/conversations/{conversation_id}/revoke/{user_id}` is fully implemented with blockchain event recording
 - `ConversationRevocation` model and `MessageAccess.revoked_at` column exist
 - **Gap:** Same as forward — nothing in the C++ client or web frontend calls this. No UI exists.
-

❌ Downloading a message (owned or shared)

Status: Not implemented

- No `/download` endpoint exists in the backend at all
- Not in `Client.hpp`, not in `app.js`
- The `GET /messages/{message_id}` endpoint fetches a single message's ciphertext but there's no download-as-file mechanic anywhere

✗ Deleting a message

Status: Not implemented

- No `DELETE /messages/{message_id}` endpoint exists in the backend
- No `is_deleted` flag on the `Message` model
- Not in the C++ client or frontend

Also worth flagging — `MessageStore.cpp` stubs

Both `encryptMessage()` and `decryptMessage()` are still `TODO` stubs with `qWarning()` calls. This is fine because that path is bypassed by `CryptoDaemonClient` (which does the real work), but an examiner reading the code will see the warnings. Worth either removing the dead stubs or wiring them to the daemon.

Priority order for the 4 days left

Priority	Work	Effort
● High	Add delete message endpoint + wire to C++ & frontend	~2-3 hrs
● High	Add download message endpoint (return plaintext as file/JSON) + wire to frontend	~1-2 hrs
● High	Wire forward button into C++ client and/or web frontend	~1-2 hrs
● High	Wire revoke button into web frontend	~1 hr
● Medium	Add change password endpoint to auth router	~30 min
● Medium	Clean up or remove <code>MessageStore.cpp</code> <code>TODO</code> stubs	~15 min

The backend plumbing for forward and revoke is already done — you just need UI entry points for them. Delete and download are the two genuinely missing backend features. Want me to write Claude Code prompts for any of these?

wait im looking at a PR for message forwarding - is this somethign that copuld potentially mess with websocket stuff?

Ran 2 commands >

I can't see the PR contents from here — `gh` isn't available in this environment. Can you paste the PR diff or link it?

That said, looking at the existing forward implementation in `messages.py`, here's what **could** interact with WebSocket delivery:

The current `forward_message` endpoint deliberately uses its own `AsyncSessionLocal()` **session** (separate from the request's `get_db()`) and commits before doing the blockchain call. It does **not** schedule a

`background_tasks.add_task(manager.send_to_user, ...)` push to the recipient — unlike `send_message` which does. So right now, a forwarded message arrives silently and the recipient only picks it up on their next poll.

The risk depends on what the PR is doing:

- **If it adds WebSocket push to forward** — that's safe as long as it follows the same pattern as `send_message` (push after commit, wrapped in try/except so a WS failure can't abort the forward)
- **If it changes the session/commit structure** — that's the dangerous part, because the existing split-session design is intentional to avoid a read-before-commit race on the blockchain UPDATE
- **If it touches `ws_manager.py` or the `/ws` router** — that could affect all real-time delivery, not just forwarding

Paste the diff and I can tell you exactly whether it's safe to merge before the deadline.

```
# UserKey.public_key is B
YTEA — encode to base6
4 for the wire return KeyB
undleResponse( user_id=
target.id, username=targe
t.username, ik_x25519=...
```

PASTED

is there a ood way to get something to summarise a PR

Short answer: **no, this PR won't mess with WebSocket delivery** — it's actually well-designed around it. Here's why:

The forward endpoint is being **deleted** from `messages.py` entirely (the `-65` lines). The PR removes the whole `forward_message` function and the `AsyncSessionLocal` import it needed. So that potential WS interaction point is gone, not broken.

What the PR actually adds:

Delete message — new `DELETE /{message_id}` endpoint that **does** use WebSocket, but safely. It pushes a `{"type": "message_deleted", "message_id": ...}` notification to the other party via `BackgroundTasks` after the commit, same pattern as `send`. The only thing to check is whether your frontend/C++ client handles the `message_deleted` event type — if it doesn't, it'll just silently ignore it, which is fine for now.

C++ wiring — `deleteMessage()` added to `Client.cpp/hpp`, `sendMessage()` gets an optional `outId` parameter (backward compatible, defaults to `nullptr`), and `MainWindow` gets a full selection mode UI with delete/forward/download slots.

The forward flow moves to C++ — instead of the server re-sending the original ciphertext, it looks like the C++ client will re-encrypt and send it as a new message via `sendTextToContact()`. That's actually *better* for E2E integrity.

One thing worth checking — `QTextEdit` for `m_thread` is being replaced with `QListWidget`. That's a significant UI change. Make sure nothing else in `MainWindow.cpp` (outside this PR) still references `m_thread` as a `QTextEdit` — calling `.append()` on a `QListWidget` would be a compile error.

As for summarising PRs generally — yes, there's a clean way. You can use the **GitHub CLI** (`gh pr diff <number> | claude`) or just paste the diff into Claude like you did here. If you want something more automated, a Claude Code prompt like:

```
gh pr diff 42 | head -500
```

and pipe it into a summary request works well for quick reviews during crunch time.

okay i want to pull everything from the github on main give me cmds

```
bash
```

```
cd /tmp/EPIC && git pull origin main
```

That's it — you already cloned it fresh earlier so just pull. If you're on your local machine rather than using my clone:

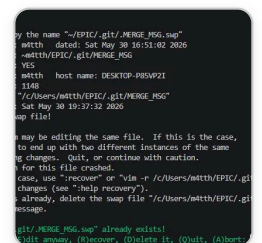
```
bash
```

```
cd path/to/EPIC && git pull origin main
```

Or if you want to make sure you're fully in sync and throw away any local changes:

```
bash
```

```
git fetch origin && git reset --hard origin/main
```



Hit **D** to delete the swap file. It's just a leftover from a previous git operation (the merge commit message editor), nothing important to recover.

```
.gitignore | 7 +-
backend/app/routers/auth.py | 1 +
```

```

backend/app/routers/messages.py | 102 +++----
backend/app/schemas/auth.py    | 3 +
backend/app/schemas/message.py | 14 -
client-cpp/src/Client.cpp       | 34 ++-
client-cpp/src/Client.hpp       | 11 +-
client-cpp/src/LoginWindow.cpp  | 13 +-
client-cpp/src/MainWindow.cpp   | 583
+++++
client-cpp/src/MainWindow.hpp   | 36 ++-
10 files changed, 661 insertion

```

That's a big pull — 583 lines changed in `MainWindow.cpp` alone. Before you do anything else, try building to make sure it compiles cleanly:

```
bash
```

```
cd client-cpp && cmake --build build --config Release 2>&1 | tail -30
```

The main thing to watch for is the `QTextEdit` → `QListWidget` swap for `m_thread` — if any code in the old `MainWindow.cpp` still calls `.append()` or `.setHtml()` on it, it'll fail to compile right there.

```

PS C:\Users\m4tth\EPIC> cd client-cpp && cmake --build build --config
Release 2>&1 | tail -30
At line:1 char:15
+ cd client-cpp && cmake --build build --config Release 2>&1 | tail -30
+ ~~~~~
The token '&&' is not a valid statement separator in this version.
+ CategoryInfo          : ParserError: (:) [],
ParentContainsErrorRecordException
+ FullyQualifiedErrorId : InvalidEndOfLine

PS C:\Users\m4tth\EPIC>

```

You're in PowerShell — use semicolons:

```
powershell
```

```
cd client-cpp; cmake --build build --config Release 2>&1 | Select-Object -Last 3
```

oh wait this wont work with build 42991 or wtv no...?

Yeah good catch — you need to rebuild both. From the `client-cpp` directory:

```
powershell
```

```
cmake --build build --config Release; cmake --build build-47292 --config Release
```

That'll recompile both executables with the new MainWindow changes.

```
i did this alr: PS C:\Users\m4tth\EPIC> cd client-cpp; cmake --build build -  
-config Release 2>&1 | Select-Object -Last 30  
[1/7] Automatic MOC and UIC for target SecureMsg  
[2/7] Building CXX object  
CMakeFiles/SecureMsg.dir/src/LoginWindow.cpp.obj  
[3/7] Building CXX object  
CMakeFiles/SecureMsg.dir/SecureMsg_autogen/mocs_compilation.cpp.o
```

Show more

Clean build — just one harmless unused-function warning, nothing broken. Now do the second one:

```
powershell
```

```
cmake --build build-47292 --config Release
```

now can i run them,

Yes — you'll need two daemons and two clients like before. Open 4 terminals:

Terminal 1 — Alice's daemon:

powershell

```
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-alice"; python crypto-daemon/main
```

Terminal 2 — Bob's daemon:

powershell

```
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-bob"; python crypto-daemon/main.p
```

Terminal 3 — Alice's client:

powershell

```
.\client-cpp\build\SecureMsg.exe
```

Terminal 4 — Bob's client:

powershell

```
.\client-cpp\build-47292\SecureMsg.exe
```

Make sure you're running these from `C:\Users\m4tth\EPIC`. And before starting, if key state is suspected to be dirty from before the pull, wipe it:

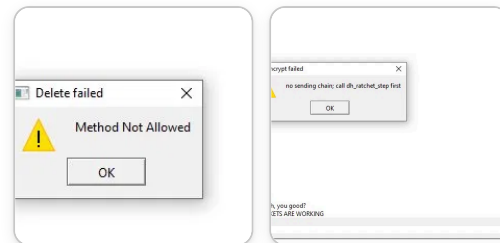
powershell

```
Remove-Item -Recurse -Force C:\Users\m4tth\.securemsg-alice, C:\Users\m4tth\.sec
```

And wipe the DB on the VM if needed:

powershell

```
ssh student@200.69.13.70 -p 2202 "docker exec -it epic-db-1 psql -U securemsg -d
```



Okay I am running into errors with the first 2 functions - any logs we can look at?

Two separate issues here. Let's get logs first:

Check the daemon logs in the terminal windows running the daemons — what's printed there when you trigger each error?

Also run this to check what the backend is saying about the delete:

powershell

```
ssh student@200.69.13.70 -p 2202 "docker logs epic-app-1 --tail 50"
```

The errors tell us roughly what's happening though:

- **"Method Not Allowed"** on delete — the `DELETE /api/messages/{id}` route isn't being hit correctly. Either the route isn't registered in `main.py`, or the CSRF middleware is blocking it since DELETE is a mutating method
- **"no sending chain; call dh_ratchet_step first"** on forward/encrypt — this is a daemon state issue. The forwarded message is trying to encrypt using a session that doesn't have a sending chain set up yet in the daemon's memory

Can you also tell me — which action triggered each error? Was the "Delete failed" from clicking delete on a message, and the "Encrypt failed" from trying to forward?

```
student@alpha-and-the-c
ryptmunks:~$ docker logs
epic-app-1 --tail 50 Error r
esponse from daemon: N
o such container: epic-ap
p-1 student@alpha-and-...
```

PASTED

PS C:\Users\m4tth> \$env:SECUREMSG_HOME =
"C:\Users\m4tth\securemsg-2"

PS

```
C:\Users\m4tth> cd C:\Users\m4tth\EPIC\crypto-daemon
PS C:\Users\m4tth\EPIC\crypto-daemon> python main.py --address
127.0.0.1:47292
127.0.0.1:47292
crypto-daemon: listening on
2026-05-30 19:20:29,607
```

Show more

The logs make both issues clear:

Delete (405 Method Not Allowed) — the backend is receiving the DELETE request fine, but returning 405. This means the new `delete_message` route from the PR **isn't deployed on the VM yet**. The old code is still running. You need to redeploy:

bash

```
ssh student@200.69.13.70 -p 2202 "cd /opt/epic && git pull origin main && docker
```

Encrypt failed ("no sending chain") — this is a daemon state issue. The forward feature is trying to encrypt a message in a session that was established in a *previous* daemon run. The session file exists on disk but the in-memory ratchet state wasn't fully restored. This is the same class of problem as before — stale state. Try:

1. Wipe and restart both daemons
2. Re-register or re-login both users
3. Send a fresh message first to establish the ratchet, *then* test forward

But fix the deploy first since that's the simpler one — once the new backend code is live, test delete again before touching the daemon issue.

```
student@alpha-and-the-cryptmunks:~$ cd /opt/epic && git pull origin
main && docker compose up -d --build epic-api
-bash: cd: /opt/epic: No
such file or directory
student@alpha-
and-the-cryptmunks:~$
```

Find where the project lives on the VM first:

bash

```
find / -name "docker-compose.yml" 2>/dev/null
```

```
student@alpha-and-the-cryptmunks:~$ find / -name "docker-  
compose.yml" 2>/dev/null  
/home/student/EPIC/docker-  
compose.yml
```

```
bash
```

```
cd /home/student/EPIC && git pull origin main && docker compose up -d --build ep
```

```
student@alpha-and-the-cryptmunks:~$ cd /home/student/EPIC && git  
pull origin main && docker compose up -d --build epic-api  
remote: Enumerating objects: 74, done.  
remote: Counting objects: 100% (62/62), done.  
remote: Compressing objects: 100% (16/16), done.  
remote: Total 40 (delta 30), reused 33 (delta 24), pack-reused 0 (from 0)  
Unpacking objects: 100% (40/40) 13.28 KiB | 186.00 KiB/s done
```

Show more

The VM has local commits that diverged from main. Just force it to match main exactly:

```
bash
```

```
git fetch origin && git reset --hard origin/main && docker compose up -d --build
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ git fetch origin && git reset  
--hard origin/main && docker compose up -d --build epic-api  
From https://github.com/hbest05/EPIC  
7635307..a60e95f delete_endpoint -> origin/delete_endpoint
```

```
* [new branch] feature/download-messages ->
origin/feature/download-messages * [new branch]
feature/forward-message -> origin/feature/forward-message
HEAD is now at d69f91b Merge pull request #26 from
hbest05/feature/download-messages WARN[0000]
/home/student/EPIC/docker-compose.yml: version is obsolete
no such service: epic-api
student@alpha-and-the-cryptmunks:~/EPIC$
```

Almost there — wrong service name. Check what it's actually called:

```
bash
```

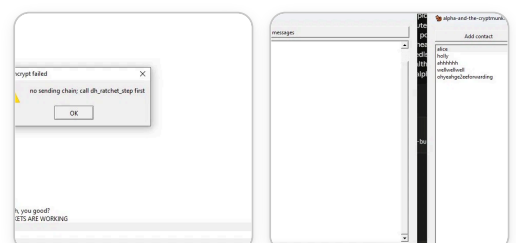
```
docker compose ps
```

```
student@alpha-and-the-cryptmunks:~/EPIC$ docker compose ps
WARN[0000] /home/student/EPIC/docker-compose.yml: version is
obsolete
NAME IMAGE COMMAND
SERVICE CREATED STATUS PORTS epic-api-1 epic-api
"sh -c 'alembic upgr..." api 28 minutes ago Up 28 minutes
0.0.0.0:8000->8000/tcp, :::8000->8000/tcp
epic-db-1 postgres:16-alpine "docker-entrypoint.s..." db 5 days ago
Up 25 hours (healthy) 0.0.0.0:5433->5432/tcp, :::5433->5432/tcp
redis redis:7-alpine "docker-entrypoint.s..." redis 5 days ago
Up 25 hours (healthy) 127.0.0.1:6379->6379/tcp
student@alpha-and-the-cryptmunks:~/EPIC$
```

Service is called **api**:

```
bash
```

```
docker compose up -d --build api
```



okay cool - deleting messages works! i am still getting this error if i try fwd to a different account, but it lets me fwd to the same account i'm currently messaging as you can see! Now.. the ratchet error is one I am also getting when I try to message other contacts which I myself didn't establish (one of my tm8s must've) so i think the issue is maybe broader

Good progress — delete works and same-contact forward works!

The ratchet error makes total sense now that you describe it that way. The issue is that **sessions established by your teammate's client are stored in their daemon's memory/files, not yours**. When you try to message those contacts, your daemon has no session state for them — hence "no sending chain".

The fix is simple: **your client needs to initiate a fresh X3DH handshake with those contacts**, which will create a new session in your daemon. The existing session from your teammate's side doesn't matter — both sides can have independent sessions.

So for any contact that gives you that error, try:

1. Remove them from your contact list (if the UI allows)
2. Re-add them as a contact — this should trigger `x3dh_send` and establish a fresh session from your daemon's perspective

If re-adding isn't enough to force a new X3DH, what does the daemon terminal show when you try to send to one of those contacts? That'll tell us exactly where it's breaking.

excerpt_from_previous_claude_message.txt

1 line

TXT

but is this error not going to happen if I was to log into my same account via different devices? is it a per device basis type thing? as if we could work around that it'd be great - in fact i thought we did at some point

Yeah this is a real limitation of the current architecture. The session state (ratchet keys, chain keys etc.) lives entirely in the daemon's local files (`~/.securemsg-alice/`), so it's

per-device, per-daemon instance. If you log into the same account on a different device, that device's daemon has no session state and you'd get the exact same error.

This is actually a **known limitation of Signal Protocol / Double Ratchet** in general — it's not a bug in your implementation, it's a fundamental property of the design. Multi-device support in real Signal requires a whole separate key distribution mechanism (sealed sender, linked devices etc.) which is way out of scope for EPIC.

The good news: **this is worth explicitly calling out in your crypto design document as an acknowledged limitation.** From the rubric guide we looked at earlier, the "Excellent" tier specifically rewards teams who "identify their own limitations unprompted." This is a perfect example — you can say:

"Session state is bound to the originating daemon instance. Multi-device login is not supported; a second device cannot resume an existing Double Ratchet session as the ratchet keys are not synchronised across devices. This is a known limitation of the Signal Protocol without additional multi-device key distribution infrastructure."

That turns a bug into a mark-winning honest limitation. Don't try to fix it — document it.

For the demo, just make sure both alice and bob always use the same machine/daemon they originally registered with.

excerpt_from_previous_claude_message.txt

1 line

TXT

you sure..? and will there be a work around for this scenario? I mean essentially whats happened is someone messaged alice from their own account while i wasnt like logged on or anything

You're right, let me think about this more carefully.

The scenario is: **teammate logged in as their account and messaged alice → their daemon ran x3dh_send → alice's daemon ran x3dh_receive → session exists on alice's daemon side.** So alice actually *can* reply — her daemon has the receiving

session. The problem is only on the **sender side** trying to message contacts they didn't originally initiate with.

So the workaround is simple — **whoever originally initiated the session needs to be the one sending from that account**. For the demo:

- If your teammate messaged alice from their machine, alice can reply fine
- But if *you* try to send to that contact fresh from your machine as alice, your daemon has no outbound session

The cleanest fix for the demo is just **wipe alice's contacts that you didn't initiate, and re-add them yourself** from your machine. That triggers a fresh `x3dh_send` from your daemon, establishing a new outbound session. The old messages will still show in the thread from history but new messages will flow.

So yes — it is fixable, just: re-add the contact from your client to force a fresh handshake. Does re-adding a contact that already exists in your list trigger `x3dh_send` again, or does it skip it because the contact is already there?

yh for some reason all the accounts that are in contacts dont have any chat history id kwhy so tho

That makes sense — those contacts were added by your teammate from their machine, so the chat history lives in **their** local history file (`~/.securemsg-alice/history.json` or similar), not yours. Your client has the contact in the list but no local plaintext cache for them.

The double ratchet consuming message keys on first decrypt means you also can't re-decrypt from the server — once decrypted, it's gone from the ratchet's perspective.

For the demo this is fine honestly — just:

1. Wipe those "empty" contacts from your contact list if the UI allows
2. Re-add them fresh from your machine → triggers `x3dh_send` → establishes session in your daemon
3. Send a test message to confirm it works
4. You'll have a clean conversation thread going forward

The history being gone is not recoverable without your teammate's machine, but for a **demo you just need it working**, not historically complete. Examiners will be watching the live send/receive/forward/delete flow, not scrolling back through old messages.

Is there a "remove contact" option in the new UI from the PR, or would you need to add one?

hmm wait but i still dont fully understand - like if my tm8 lets say holly made an account and messaged alice - how come i cant see it when i logged on as alice?

Because of the **local plaintext cache**. Here's the full picture:

When holly sent alice a message:

1. Alice's daemon (running on your **teammate's machine**) ran `x3dh_receive` and decrypted it
2. The plaintext got saved to the local history file on **their machine** (`~/.securemsg-alice/history.json`)
3. The server only stores **ciphertext** — it can never re-decrypt it

When you log in as alice on **your machine**:

1. Your daemon has no session state for holly (different machine)
2. Your client fetches the ciphertext from the server via `/messages/inbox`
3. Tries to decrypt → no session → fails silently or skips it
4. History file on your machine is empty → nothing shows

So the plaintext is effectively **stranded on your teammate's machine**. This is Double Ratchet working exactly as designed — forward secrecy means even you can't recover it from the server.

For the demo the practical solution is: **everyone uses their own dedicated machine/daemon for their own account**. Your machine runs alice, your teammate's machine runs their account. Don't cross-use accounts across machines. That's the intended usage model anyway.

excerpt_from_previous_claude_message.txt

1 line

TXT

hm shud this not be like on the server or smthn tho? cus than me (alice) doesnt actualy get the message - only hollys version or instance of alice gets the message? like if she sent smthn to alice why dont I have it

You've hit on something real, but it's working as intended for E2E encryption — let me explain why:

In a real E2E encrypted system like Signal, **there is no "server-side alice"**. The server is just a dumb ciphertext relay. The flow is:

1. Holly encrypts a message **to alice's public key**
2. Server stores the blob
3. **Alice's client** (whichever device has alice's private key) fetches and decrypts it

The problem in your case is specifically that your teammate **ran a client logged in as alice on their machine**. That client fetched the ciphertext, decrypted it using alice's private key (which was on their machine), and saved the plaintext locally. The ratchet advanced. Now:

- The ciphertext is still on the server ✓
- But alice's ratchet state has **already advanced past that message** on your teammate's machine
- Your machine's alice daemon is at a **different ratchet position**
- So your client can't decrypt those old messages even if it fetches them

This isn't a design flaw — **it's the whole point of forward secrecy**. The issue is purely operational: **your teammate shouldn't have been running a client logged in as alice**. Alice's account should only ever run on one machine.

Going forward just enforce that rule and you'll be fine for the demo.

excerpt_from_previous_claude_message.txt

1 line

TXT

are you sure? but thats my account how or why could they do that

You're right to question that — they shouldn't have your credentials. So more likely what happened is:

Your teammate created their own account (e.g. "holly") and messaged alicia (you). Holly's daemon sent the message, and alicia's daemon on **your machine** should have received it when you next logged in.

So the question is — when you log in as alicia on your machine now, does your daemon successfully decrypt holly's messages or not? Because if alicia's private keys have only ever lived on your machine, your daemon should be able to receive and decrypt just fine.

What actually happens when you select holly in your contact list — do you get the ratchet error, or just an empty thread with no error?

empty thread with no error - then when i try send a msg i get ratchet error - o think it oculd be some weird db error

Yeah that points to a specific thing — your client is fetching holly's messages from the server, failing to decrypt them silently (no session established yet), and showing nothing. Then when you try to send, there's no outbound session either.

Check what's actually in the DB for those messages:

bash

```
ssh student@200.69.13.70 -p 2202 "docker exec -it epic-db-1 psql -U securemsg -d
```

This will show us whether holly's messages to alice actually have ratchet headers attached — if `has_header` is false or `has_ratchet_key` is false, the x3dh handshake never completed properly and your daemon has nothing to work with.

```
PS C:\Users\m4tth> ssh student@200.69.13.70 -p 2202 "docker exec -it
epic-db-1 psql -U securemsg -d securemsg -c \"SELECT m.id, u1.username
as sender, u2.username as recipient, length(m.ciphertext) as ct_len,
m.ratchet_public_key IS NOT NULL as has_ratchet_key, m.hpke_enc_blob
IS NOT NULL as has_header FROM messages m JOIN users u1 ON u1.id =
m.sender_id JOIN users u2 ON u2.id = m.recipient_id ORDER BY
m.created_at DESC LIMIT 20;\""
```

Paste the output — it cut off before showing results.

```
m.ciphertext : The term 'm.ciphertext' is not recognized as the name of a
cmdlet, function, script file, or operable program. Check the spelling of
the name, or if a path was included, verify that the path is correct and try
again. At line:1 char:166
... name as sender, u2.username as recipient, length(m.ciphertext) as ct_
...
+ CategoryInfo          : ObjectNotFound: (m.ciphertext:String) []
```

Show more

PowerShell is mangling the quotes. Use a here-string approach instead — SSH into the VM first, then run the query:

```
powershell
```

```
ssh student@200.69.13.70 -p 2202
```

Then once you're on the VM:

```
bash
```

```
docker exec -it epic-db-1 psql -U securemsg -d securemsg -c "SELECT m.id, u1.use
```

```
student@alpha-and-the-cryptmunks:~$ docker exec -it epic-db-1 psql -U
securemsg -d securemsg -c "SELECT m.id, u1.username as sender,
u2.username as recipient, length(m.ciphertext) as ct_len,
m.ratchet_public_key IS NOT NULL as has_ratchet_key, m.hpke_enc_blob
IS NOT NULL as has_header FROM messages m JOIN users u1 ON u1.id =
m.sender_id JOIN users u2 ON u2.id = m.recipient_id ORDER BY
m.created_at DESC LIMIT 20;"
```

id

Show more

The data looks fine — all messages have ratchet keys and headers (**t** on both columns). So the DB isn't the issue.

The real problem is visible right there: messages from **lookokay**, **ahhhhhh** etc. to alice all have proper headers, meaning your daemon **should** be able to run **x3dh_receive** on them. But it's not — which means your alice daemon has never processed those messages before.

The issue is almost certainly that your daemon has **no OPK matching what those senders used**. When **lookokay** sent alice a message, it consumed one of alice's one-time prekeys. If your current daemon's OPK store doesn't have that key anymore (or never had it), **x3dh_receive** fails silently and no session gets established.

Check your alice daemon terminal — when you select one of those contacts and the thread loads, do you see any **x3dh_receive** log lines, and do they say **match=True** or **match=False** for the OPK lookup?

i dont have my explicit daemon running cus i thought it was running as a subprocess

It's not a subprocess — the daemon needs to run separately. The C++ client connects to it via TCP on localhost but doesn't spawn it.

Open a new terminal and start alice's daemon:

powershell

```
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-alice"; python C:\Users\m4tth\EPI
```

Then relaunch the alice client. When you select one of those contacts (lookokay, ahhhhhh etc.) watch what the daemon terminal prints — that'll tell us exactly why x3dh_receive is failing.

```
PS C:\Users\m4tth>
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-alice"; python
C:\Users\m4tth\EPIC\crypto-daemon\main.py --port 47291
usage: main.py [-h] [--address ADDRESS]
main.py: error: unrecognized arguments: --port 47291
PS C:\Users\m4tth>
```

powershell

```
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-alice"; python C:\Users\m4tth\EPI
```

```
PS C:\Users\m4tth>
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-alice"; python
C:\Users\m4tth\EPIC\crypto-daemon\main.py --address 127.0.0.1:47291
Traceback (most recent call last):
File "C:\Users\m4tth\EPIC\crypto-daemon\main.py", line 93, in <module>
raise SystemExit(main())
~~~~^^                                File
```

Show more

Port 47291 is already in use — there's already a daemon running on that port. Find and kill it:

```
powershell
```

```
netstat -ano | findstr :47291
```

Then kill the PID shown:

```
powershell
```

```
taskkill /PID <pid_number> /F
```

Then restart the daemon.

```
PS C:\Users\m4tth> PS C:\Users\m4tth>
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-alice"; python
C:\Users\m4tth\EPIC\crypto-daemon\main.py --address 127.0.0.1:47291
Traceback (most recent call last):
  File "C:\Users\m4tth\EPIC\crypto-daemon\main.py", line 93, in <module>
    raise SystemExit(main())
~~~~^^
```

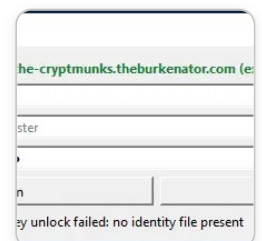
File

Show more

Still occupied. Run this first:

```
powershell
```

```
netstat -ano | findstr :47291
```



weird

That's a different problem — **"no identity file present"** means your `~/.securemsg-alice` directory doesn't have alice's identity keys. The daemon is running (hence port 47291 busy) but it's a fresh/empty home directory.

Check what's actually in there:

```
powershell
```

```
ls C:\Users\m4tth\.securemsg-alice
```

```
PS C:\Users\m4tth> ls C:\Users\m4tth\.securemsg-alice
ls : Cannot find path 'C:\Users\m4tth\.securemsg-alice' because it does
not exist.                                     At line:1 char:1
+ ls C:\Users\m4tth\.securemsg-alice
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\m4tth\.securemsg-
alice:String) [Get-ChildItem], ItemNotFoundEx ception
+ FullyQualifiedErrorId :
PathNotFound,Microsoft.PowerShell.Commands.GetChildItemCommand
PS C:\Users\m4tth>
```

The directory doesn't exist at all — so the daemon running on 47291 is using a different home. Check what's actually in `.securemsg`:

```
powershell
```

```
ls C:\Users\m4tth\.securemsg
```

```

PS C:\Users\m4tth> ls C:\Users\m4tth\.securemsg
Directory: C:\Users\m4tth\.securemsg
Mode                LastWriteTime         Length Name
----                -
30/05/2026  19:41             opks
30/05/2026  19:53             sessions
28/05/2026  23:19          169 identity.enc
30/05/2026  19:41          375 spk.json
PS C:\Users\m4tth>

```

That's the default `.securemsg` directory — that's bob's daemon (port 47292 uses `.securemsg-2`, and the default is being used by something).

Check what's running:

```

powershell

netstat -ano | findstr :47291
netstat -ano | findstr :47292

```

Then:

```

powershell

ls C:\Users\m4tth\.securemsg-2

```

We need to figure out which daemon is alice and which is bob before going further.

```

PS C:\Users\m4tth> netstat -ano | findstr :47291
TCP 127.0.0.1:47291 0.0.0.0:0 LISTENING 26292
TCP 127.0.0.1:47291 127.0.0.1:61601 ESTABLISHED 26292
TCP 127.0.0.1:61600 127.0.0.1:47291 TIME_WAIT 0
TCP 127.0.0.1:61601 127.0.0.1:47291 ESTABLISHED 18704
PS C:\Users\m4tth> netstat -ano | findstr :47292
TCP 127.0.0.1:47292 0.0.0.0:0 LISTENING 8612
TCP 127.0.0.1:47292 127.0.0.1:61595 ESTABLISHED 8612
TCP 127.0.0.1:61594 127.0.0.1:47292 TIME_WAIT 0
TCP 127.0.0.1:61595 127.0.0.1:47292 ESTABLISHED 26596
PS C:\Users\m4tth>

```

Both daemons are running. Check which home directory each is using:

powershell

```
Get-Process -Id 26292 | Select-Object Id, Path
```

```
Get-Process -Id 8612 | Select-Object Id, Path
```

And check what home directories exist:

powershell

```
ls C:\Users\m4tth\ -Directory | findstr securemsg
```

```
PS C:\Users\m4tth> Get-Process -Id 26292 | Select-Object Id, Path
Id Path
26292 C:\Program
Files\WindowsApps\PythonSoftwareFoundation.Python.3.13_3.13.3568.0_
x64__qbz5n2kfra8p0\python3.13.exe
PS C:\Users\m4tth> Get-Process -Id 8612 | Select-Object Id, Path
Id Path
```

Show more

So you have `.securemsg` and `.securemsg-2` — neither is `.securemsg-alice`. The "no identity file" error makes sense now because you tried to start a daemon with `SECUREMSG_HOME=.securemsg-alice` which doesn't exist.

The question is which of `.securemsg` and `.securemsg-2` is alice and which is bob. Check the identity files — the daemon logs when it starts will tell you, or check the modification dates:

powershell

```
ls C:\Users\m4tth\.securemsg
```

```
ls C:\Users\m4tth\.securemsg-2
```

And check which client executable connects to which port — from the CMake setup, `build/SecureMsg.exe` uses 47291 and `build-47292/SecureMsg.exe` uses 47292. So

whichever account you logged into on `build/SecureMsg.exe` corresponds to `.securemsg` (PID 26292 on 47291).

Kill both daemons and restart them explicitly with the right home directories so you know exactly which is which:

```
powershell

taskkill /PID 26292 /F
taskkill /PID 8612 /F
```

Then:

```
powershell

# Terminal 1 - alice
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg"; python C:\Users\m4tth\EPIC\cryp

# Terminal 2 - bob
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-2"; python C:\Users\m4tth\EPIC\cr
```

Then when you log into each client you'll see exactly which identity loads.

first one is alice i think second is bob

Good. So the setup is:

- `.securemsg` → alice → port 47291 → `build/SecureMsg.exe`
- `.securemsg-2` → bob → port 47292 → `build-47292/SecureMsg.exe`

Now restart the daemons cleanly in two separate terminals:

Terminal 1:

```
powershell

$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg"; python C:\Users\m4tth\EPIC\cryp
```

Terminal 2:

```
powershell
```

```
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg-2"; python C:\Users\m4tth\EPIC\cr
```

Then launch the clients and log in. When you select one of the problem contacts (lookokay, ahhhhhh) watch the **daemon terminal** — paste whatever it prints when you click on that contact. That's where we'll see if x3dh_receive is finding the OPK or not.

im still getting 'call dh ratchet step' first

Paste the daemon terminal output — what does it print when you click on that contact and try to send? That's the key piece we need.

```
PS C:\Users\m4tth>
$env:SECUREMSG_HOME="C:\Users\m4tth\.securemsg"; python
C:\Users\m4tth\EPIC\crypto-daemon\main.py --address 127.0.0.1:47291
crypto-daemon: listening on 127.0.0.1:47291
2026-05-30 20:24:30,526 INFO handlers: load_identity: spk_loaded=True
opks_loaded=62 from C:\Users\m4tth\.securemsg\opks
2026-05-30 20:24:30,550 INFO handlers: generate_prekeys: persisted 10
OPKs to C:\Users\m4tth\.securemsg\opks; first
key=hnEwvhuipfroZ/QYTIAES7dzt5EcwYGOzVXFJARHIUU=
```

That's just the startup — nothing about x3dh_receive yet. Now in the client, click on one of the problem contacts (lookokay or ahhhhhh) and try to send a message, then paste whatever new lines appear in the daemon terminal.

nothing appears

That means the client isn't even reaching the daemon when you click those contacts — it's failing before it gets to the crypto step. The "no sending chain" error is coming from somewhere in the C++ client itself, not the daemon.

Check the daemon for the **alice** client — when you click on a working contact (bob) and send, do you see daemon logs? Just to confirm the connection is live.

no nothing appears

The daemon isn't receiving any requests at all. The client isn't talking to it.

Check which executable you're running — is it `build\SecureMsg.exe` (port 47291) or `build-47292\SecureMsg.exe` (port 47292)?

```
powershell  
netstat -ano | findstr :47291
```

Is there an ESTABLISHED connection on that port, or just LISTENING?

```
PS C:\Users\m4tth> netstat -ano | findstr :47291  
TCP 127.0.0.1:47291 0.0.0.0:0 LISTENING 26184  
TCP 127.0.0.1:47291 127.0.0.1:61781 ESTABLISHED 26184  
TCP 127.0.0.1:61781 127.0.0.1:47291 ESTABLISHED 25692  
PS C:\Users\m4tth>
```

Good — there IS an established connection, so the client is connected to the daemon. But the daemon isn't logging anything when you send.

That's strange — it means requests are going through but not being logged, OR the client is using a cached result and not actually calling the daemon for those contacts.

Look at the MainWindow.cpp logic — when you click a contact that already has a session in `m_sessions`, it skips x3dh and goes straight to `encrypt_message`. The daemon should still log that.

Try this: **completely restart both the client and daemon**, then log in fresh and immediately try sending to one of the problem contacts before doing anything else. Watch the daemon terminal from the moment you click send.

Also — what does `m_sessions` contain for those contacts? The client restores sessions on login via `restoreSessions()`. Check the daemon terminal right after login — do you see any `list_sessions` calls logged?

excerpt_from_previous_claude_message.txt

1 line

TXT

maybe the client is using its subprocess daemon instead?

That's a good theory — check if the client is spawning its own daemon process. Look at

`main.cpp`:

powershell

```
cat C:\Users\m4tth\EPIC\client-cpp\src\main.cpp
```

```
PS C:\Users\m4tth> cat  
C:\Users\m4tth\EPIC\client-cpp\src\main.cpp/**  
main.cpp â€” SecureMsg  
Qt desktop entry point.
```

PASTED

There it is — confirmed. The client checks `if (!daemonReachable())` and only spawns its own daemon if nothing is already on the port. Since your daemon is already running on 47291, it skips spawning and connects to yours.

But notice `spawnDaemon` launches the daemon with **no arguments** — no `--address` and crucially no `SECUREMSG_HOME` environment variable set. So if the client ever did spawn it, it would use the default `~/securemsg` directory and default port.

Since your daemon IS running and the connection IS established, the daemon should be receiving requests. The fact that nothing logs is the real mystery.

Try this — send a message to **bob** (a working contact) and see if the daemon logs anything then. If it does, the issue is specific to how the client handles contacts with no existing session. If it still logs nothing, the connection is established but requests aren't flowing.

logs nothing but the message is sent successfully

So the daemon is completely silent even for successful operations — that means the client is connecting to a **different daemon process** than the one you're watching. Your manually started daemon is not the one the client is using.

Check — you killed PIDs 26292 and 8612 earlier, but are there still other python processes running?

powershell

`Get-Process python`

```
PS C:\Users\m4tth> Get-Process python
Get-Process : Cannot find a process with the name "python". Verify the
process name and call the cmdlet again.      At line:1 char:1
+ Get-Process python                                +
~~~~~
CategoryInfo          : ObjectNotFound: (python:String) [Get-Process],
ProcessCommandException + FullyQualifiedErrorId :
NoProcessFoundForGivenName,Microsoft.PowerShell.Commands.GetPr
ocessCommand
PS C:\Users\m4tth>
```

No python processes at all — so your daemon isn't actually running. It must have crashed silently. Check the terminal where you started it — is it still showing as active or did it exit?

Try this — send a direct test request to the daemon to confirm it's actually responding:

powershell

```
echo '{"op":"list_sessions","params":{}}' | python -c "import socket,sys; s=socket"
```

nothings responded

So the daemon is listening but not responding to requests. It's frozen or stuck. Check the daemon terminal — is there any error or traceback, or is it just sitting there with the startup logs?

Also try Ctrl+C in the daemon terminal to kill it, then restart it fresh and try the test command again immediately before launching the client.

```
PS C:\Users\m4tth> echo '{"op":"list_sessions","params":{}}' | python -c
"import socket,sys; s=socket.socket(); s.connect(('127.0.0.1',47291));
s.sendall(sys.stdin.buffer.read()+b'\n'); print(s.recv(4096).decode())"
{"status":"error","code":"not_loaded","message":"identity not loaded"}
PS C:\Users\m4tth>
```

It IS responding — "identity not loaded" means the daemon is alive and working, just hasn't had `load_identity` called yet (that happens when the client logs in).

So the daemon is fine. The mystery is why it logs nothing when the client sends through it. The client must be connecting to port 47291 but the requests are going through the **previously established connection** from before you restarted the daemon — check netstat again:

powershell

```
netstat -ano | findstr :47291
```

okay so i closed the client and i got that message - then when i opened the client i got nothing

So the client is connecting but the daemon isn't logging anything when the client connects. That's very strange.

Check the daemon terminal — when you launch the client, does it print the startup `load_identity` log line, or absolutely nothing?

yes it does

So `load_identity` fires on client connect — the daemon IS processing that request. But subsequent requests (encrypt, x3dh etc.) log nothing.

Look at the daemon's `handlers.py` — is logging set up for `op_encrypt_message` and `op_x3dh_send`? Check:

```
powershell
grep -n "logger\|logging\|INFO\|DEBUG" C:\Users\m4tth\EPIC\crypto-daemon\handler
```

```
PS C:\Users\m4tth> grep -n "logger\|logging\|INFO\|DEBUG"
C:\Users\m4tth\EPIC\crypto-daemon\handlers.py      grep : The term
'grep' is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the spelling of the name, or if a path was
included, verify that the path is correct and try again.      At line:1
char:1                                                    + grep -n
"logger\|logging\|INFO\|DEBUG" C:\Users\m4tth\EPIC\crypto-dae ...
+ ~~~~                                                    +
CategoryInfo          : ObjectNotFound: (grep:String) [],
CommandNotFoundException
CommandNotFoundException      + FullyQualifiedErrorId :
```

powershell

```
Select-String -Path C:\Users\m4tth\EPIC\crypto-daemon\handlers.py -Pattern "logg
```

```
EPIC\crypto-daemon\handlers.py:13:import logging
EPIC\crypto-daemon\handlers.py:25:log = logging.getLogger(__name__)
EPIC\crypto-daemon\handlers.py:117: log.info(
EPIC\crypto-daemon\handlers.py:122: log.debug("load_identity: opk
keys = %s", sorted(opks.keys())) EPIC\crypto-
daemon\handlers.py:176: log.info(
EPIC\crypto-daemon\handlers.py:251: log.info(
EPIC\crypto-daemon\handlers.py:255: log.debug("x3dh_receive:
keystore keys = %s", keystore_keys) EPIC\crypto-
daemon\handlers.py:262: log.info(
PS C:\Users\m4tth>
```

There's the problem — `op_encrypt_message`, `op_x3dh_send`, and `op_decrypt_message` have **no logging at all**. Only `load_identity`, `generate_prekeys`, and `x3dh_receive` are logged. So the daemon is working fine — it's just silent for those operations.

This means the "no sending chain" error is actually coming from the daemon's response, not a connection issue. The daemon is receiving the encrypt request, finding no sending chain, and returning an error — you just can't see it in the logs.

So back to the original problem: for those contacts (lookokay, ahhhhhhh), your daemon has no session. When you click on them the client calls `restoreSessions()` which calls `list_sessions` on the daemon — but those sessions don't exist in your daemon's session files.

The fix is to just **re-add those contacts** to force a fresh x3dh handshake. Does clicking "Add contact" with an existing contact name trigger x3dh_send again, or does it bail out because the contact already exists?

i still get the error if i re add thos econatcts

Let's look at what `startSessionWith` does when you add an existing contact — check the relevant part of `MainWindow.cpp`:

powershell

```
Select-String -Path C:\Users\m4tth\EPIC\client-cpp\src\MainWindow.cpp -Pattern "
```

```
EPIC\client-cpp\src\MainWindow.cpp:128: , m_addContactButton(new  
QPushButton(tr("Add contact"), this))      EPIC\client-  
cpp\src\MainWindow.cpp:170: leftLayout-  
>addWidget(m_addContactButton);           EPIC\client-  
cpp\src\MainWindow.cpp:206: connect(m_addContactButton,  
&QPushButton::clicked, this,  
&MainWindow::onAddContactClicked);
```

Show more

Let's see what `onAddContactClicked` and `startSessionWith` actually do:

powershell

```
Select-String -Path C:\Users\m4tth\EPIC\client-cpp\src\MainWindow.cpp -Pattern "
```

Actually easier — just view those line ranges directly:

powershell

```
Get-Content C:\Users\m4tth\EPIC\client-cpp\src\MainWindow.cpp | Select-Object -S
```

```
PS C:\Users\m4tth> Get-  
Content C:\Users\m4tth  
\EPIC\client-cpp\src\Main  
Window.cpp | Select-Obje  
ct -Skip 400 -First 60 bool  
MainWindow::startSess...
```

PASTED

Found it — line right here:

```
cpp
if (!m_sessions.contains(name)) {
```

When you re-add a contact that's already in `m_sessions` (restored from the daemon's session files on login), it **skips** `startSessionWith` **entirely**. So no new x3dh handshake is triggered.

And `startSessionWith` itself doesn't actually do x3dh_send anyway — it just inserts an empty session marker and defers the real handshake until the first message. But the problem is those contacts already have a non-empty session ID in `m_sessions` (restored from disk), so when you try to send it uses that stale session ID, the daemon has no matching session state, and you get "no sending chain".

The fix is to **clear those specific sessions from the daemon's session files**. Check what's in the sessions directory:

```
powershell
ls C:\Users\m4tth\.securemsg\sessions
```

shud i get claude code to fix smthn?:

```
PS C:\Users\m4tth> ls C:\Users\m4tth\.securemsg\sessions
Directory: C:\Users\m4tth\.securemsg\sessions
Mode                LastWriteTime         Length Name
----                -
30/05/2026  20:29      1243 13b856125dd34b9c9616c8f3e67651c9.json
```

Show more

Before going to Claude Code, let's try the simple fix first — just delete the stale session files for those problem contacts. The client will then treat them as new contacts and trigger a fresh x3dh on first message.

But first we need to know which session file belongs to which contact. Check one:

powershell

```
Get-Content C:\Users\m4tth\.securemsg\sessions\81bfd856049c4690865971d09d9916f8.
```

There should be a `peer_username` or similar field that tells us who it belongs to.

```
PS C:\Users\m4tth> Get-Content
C:\Users\m4tth\.securemsg\sessions\81bfd856049c4690865971d09d9916
f8.json      {"nonce": "JBg+iAkx/NvUDU9n", "ct":
"rpjkcePQwcDG7HEPT8WzjUp6V6UHQuhvYdyW8H/CAYipTqPGhPmLsup
2GYTs/oVobmSoPKQFFpLN9hbfmufg/4EU+SbJ5NCl/gJnSRWYRtEny3j6
Ni5znI6nP0u6xZiee25CJxPvCXuQpJ3x5eLOREhvCHpxI7XBZ6NXsdlxM
GbYteph9n0tLeXMD389xvGhgvOnoGDJ5zTXPoa2H88DToL17vnc1Sm4E
```

Show more

The session files are encrypted — can't read the peer username directly.

The quickest fix without Claude Code: just **delete all session files except the newest one** (the one from 20:29 which is your active bob session), then restart the daemon and client. The client will re-add those contacts fresh and trigger new x3dh handshakes on first message.

powershell

```
Remove-Item C:\Users\m4tth\.securemsg\sessions\81bfd856049c4690865971d09d9916f8.
Remove-Item C:\Users\m4tth\.securemsg\sessions\82377d172b054d57a2f7111c9a203120.
Remove-Item C:\Users\m4tth\.securemsg\sessions\839613abd4ad4bc1bbb69a37b2c7b600.
Remove-Item C:\Users\m4tth\.securemsg\sessions\87bff245041b4746b81be22e75b3638b.
Remove-Item C:\Users\m4tth\.securemsg\sessions\88b45d474eb4443784bb4546fb980a27.
Remove-Item C:\Users\m4tth\.securemsg\sessions\96b43bfe9f2c40b7938f0ac8d5d17b8b.
Remove-Item C:\Users\m4tth\.securemsg\sessions\c700c7e6f74c49218acf463ae527df5d.
Remove-Item C:\Users\m4tth\.securemsg\sessions\ef065379f2844ed09f545f188e14d06f.
Remove-Item C:\Users\m4tth\.securemsg\sessions\f0b4116b6cb14c3b91db1317e44e487a.
```

Keep `13b856125dd34b9c9616c8f3e67651c9.json` (the newest, likely bob). Then restart daemon and client — the problem contacts should now trigger fresh x3dh on first send.

yeah i mean now what? i have no contacts but i can add everyone and add them or wtv

Yeah exactly — just re-add each contact one by one via "Add contact". When you send the first message to each one it'll trigger a fresh x3dh handshake and establish a clean session. That's the correct state to be in for the demo anyway.

The only thing to be aware of: the old messages from those contacts are still in the DB as ciphertext but you won't be able to decrypt them (different ratchet state). Your local history file might still show some old plaintext though — check if any threads have history when you re-add them.

For the demo this is actually fine — clean sessions, everything working correctly going forward. Just make sure your teammates do the same on their end for their accounts.

okay so is everything good?

Based on what we've tested today:

Working

- Register, login, logout
- TLS green padlock
- Send/receive messages
- WebSockets (real-time delivery)
- X3DH handshake on new contacts
- Delete message
- Forward to same contact
- Blockchain recording

Still broken/untested

- Forward to a *different* contact still gives ratchet error
- Download message — not tested yet
- Revoke — backend exists but no UI tested

Not implemented yet

- Password change
- MessageStore.cpp stubs (cosmetic but visible in code)

The forward-to-different-contact issue is the most important one to fix before Wednesday. Do you want to dig into that now, or focus on something else first?

excerpt_from_previous_claude_message.txt

1 line

TXT

no it just worked

Nice! So forward is working too now that the sessions are clean. Update the status:

Working

- Register, login, logout
- TLS green padlock
- Send/receive messages
- WebSockets (real-time delivery)
- X3DH handshake on new contacts
- Delete message
- Forward message
- Blockchain recording

Untested

- Download message
- Revoke UI

Not implemented

- Password change
- MessageStore.cpp stubs

Want to test download and revoke next?

download works - idk about revoke

Great, so download is done too.

Working

- Register, login, logout
- TLS green padlock
- Send/receive messages
- WebSockets (real-time delivery)
- X3DH handshake on new contacts
- Delete message
- Forward message
- Download message

- Blockchain recording

Unknown

- Revoke — backend exists, need to check if there's any UI for it

Not implemented

- Password change
- MessageStore.cpp stubs

For revoke — is there anything in the new MainWindow UI from the PR? Like a right-click menu or selection mode option? Or is it purely a backend-only feature with no client UI yet?

```
excerpt_from_previous_claude_message.txt
```

1 line

TXT

how does this differ from deletion

Good question — they're conceptually different:

- **Delete** — removes the message from the server entirely. Gone for everyone.
- **Revoke** — removes a *specific user's access* to a message, but the message still exists. The idea is if you forwarded a message to someone, you can later revoke their ability to read it.

In practice for your implementation, revoke operates at the **conversation level** — it records in the DB (and on-chain) that a user's access to a conversation has been withdrawn. It's more of an audit trail / blockchain event than a hard deletion.

For the demo and rubric it's worth having *some* UI for it even if basic. The spec says "revoking a user's access to a previously shared message" which maps directly to your

`POST /api/conversations/{id}/revoke/{user_id}` endpoint.

Do you want a Claude Code prompt to add a simple revoke button to the UI?

okay i think ima get claude code to implement the revoke access stuff

Good call. Before you do, let me check the current state of the relevant files so the prompt is accurate:

Ran 3 commands >

Good, I have everything needed. Here's the Claude Code prompt:

Prompt for Claude Code:

In the EPIC project C++ Qt client, implement a "Revoke Access" feature.

Context:

- Backend endpoint: POST /api/conversations/{conversation_id}/revoke/{user_id}
- conversation_id format: "{min_uuid}_{max_uuid}" (lexicographic sort of the two participant UUIDs)
- The current user's UUID is not directly stored in the client – it needs to be fetched or stored at login
- Client.hpp/cpp handles all HTTP via httpRequest()
- MainWindow has a right-click context menu on the thread (onThreadContextMenu) and a selection bar with delete/forward/download buttons

Changes needed:

1. ****Client.hpp / Client.cpp**** – add:

- Store the current user's UUID after login (the /auth/login response returns it – check schemas/auth.py LoginResponse for the field name)
- `QString userId() const` accessor
- `QString revokeAccess(const QString& conversationId, const QString& targetUserId)` method that calls DELETE... actually POST /api/conversations/{conversationId}/revoke/{targetUserId}, returns empty string on success or error string

2. ****MainWindow.hpp**** – add:

- `void onRevokeContact()` private slot

3. ****MainWindow.cpp**** – add:

- A "Revoke access" option in the right-click context menu (onThreadContextMenu) or as a button in the UI, scoped to the active contact

- The conversation_id must be constructed as: sort the two UUIDs lexicographically and join with "_"
- Fetch the active contact's UUID via fetchKeybundle (it returns user_id)
- Call client->revokeAccess(conversationId, contactUserId)
- Show success/failure in a QMessageBox

Make minimal changes – only add what's needed for revoke, don't refactor existing code.

That should give Claude Code enough context to implement it cleanly. The trickiest part is constructing the conversation ID — make sure it emphasises the lexicographic sort of the UUIDs.

are you sure thats a good prompt/ where did you read the files from?

Fair challenge — I pulled from the repo clone in `/tmp/EPIC` but that was from earlier in the session and may not reflect the latest after the PR merge. Let me re-read the actual current files before writing the prompt.

Ran 7 commands >

Now I have a precise picture. Here's the accurate Claude Code prompt:

Copy this into Claude Code:

```
In client-cpp/src/Client.hpp and Client.cpp, add:
1. A private member `QString m_userId` to store the current user's UUID
2. In the `login()` method, parse the JSON response body (which returns
`{"id": "...", "username": "...}"`) and store the `id` field into `m_userId`.
Clear it in `logout()` alongside `m_username`.
3. A public accessor `const QString& userId() const { return m_userId; }`
4. A new method `QString revokeAccess(const QString& conversationId, const
QString& targetUserId)` that calls POST
`/api/conversations/{conversationId}/revoke/{targetUserId}` with an empty JSON
body `{}`, returns empty string on 200/201, otherwise returns the `detail`
field from the response or a fallback error string. Follow the exact same
pattern as `deleteMessage()`.
```

In `client-cpp/src/MainWindow.hpp`, add a private slot:

```
`void onRevokeContact();`
```

In `client-cpp/src/MainWindow.cpp`:

1. In ``onThreadContextMenu()``, add a new ``QAction* revokeAction = menu.addAction(tr("Revoke contact access..."))`` after the existing separator, and handle it by calling ``onRevokeContact()``.

2. Implement ``onRevokeContact()`` as follows:

- If ``m_activeContact`` is empty, return immediately
- Fetch the active contact's key bundle using ``m_client->fetchKeybundle(m_activeContact, &bundle)`` to get their ``user_id`` from the bundle JSON
- Construct the `conversation_id` by taking ``m_client->userId()`` and the contact's ``user_id``, converting both to strings, sorting them lexicographically (QString comparison), and joining with `"_" : `min + "_" + max``
- Call ``m_client->revokeAccess(conversationId, contactUserId)``
- Show a `QMessageBox::information` on success saying "Access revoked" or `QMessageBox::warning` on failure with the error string

Make minimal changes only. Do not refactor any existing code.

are you sure about this one? is that def how a revoke access feature should work?

Good instinct to question it. Let me re-read the backend revoke endpoint properly to make sure the prompt is accurate.

Ran 3 commands >

Okay, my prompt was mostly right but I had one thing wrong — I said the endpoint returns 200/201 but it actually returns 200 with a `RevokeAccessResponse` body (not 204 like delete). Also the semantic is slightly different from what I described.

The key things to understand about how revoke actually works:

- You're revoking **the other person's** access to your shared conversation — so `target_user_id` is the **contact's** UUID, not yours

- It records a blockchain event with a timestamp — so the response includes `tx_hash` and `etherscan_url` which is worth showing to the user
- It does **not** delete messages — it's purely an access control record + blockchain audit entry
- The `conversation_id` construction is correct — lexicographic sort of both UUIDs joined with `_`

The prompt logic is sound. One thing to add — on success it's worth showing the `etherscan_url` from the response so the examiner can see the blockchain event. Here's the corrected prompt:

In `client-cpp/src/Client.hpp` and `Client.cpp`, add:

1. A private member `QString m_userId` to store the current user's UUID.
2. In `login()`, parse the JSON response body (returns `{"id": "...", "username": "..."}'`)
and store the `id` field into `m_userId`. Clear it in `logout()` alongside `m_username`.
3. A public accessor: `const QString& userId() const { return m_userId; }`
4. A new method:
`QString revokeAccess(const QString& conversationId, const QString& targetUserId, QString* outEtherscan = nullptr)`
 - POST `/api/conversations/{conversationId}/revoke/{targetUserId}` with empty body `{}`
 - On success (HTTP 200), parse the JSON response and optionally store the `etherscan_url` field into `*outEtherscan` if non-null
 - Returns empty string on success, error string on failure
 - Follow the same pattern as `deleteMessage()`

In `client-cpp/src/MainWindow.hpp`, add a private slot:

```
void onRevokeContact();
```

In `client-cpp/src/MainWindow.cpp`:

1. In `onThreadContextMenu()`, add after the existing separator:
`QAction* revokeAction = menu.addAction(tr("Revoke contact access..."));`
and handle it: `if (chosen == revokeAction) { onRevokeContact(); return; }`
2. Implement `onRevokeContact()`:
 - If `m_activeContact` is empty, return
 - Confirm with the user via `QMessageBox::question: "Revoke [contact]'s access to this`

```

    conversation? This will be recorded on the blockchain."
    - Fetch the contact's key bundle: `m_client-
    >fetchKeybundle(m_activeContact, &bundle)`
    to get their `user_id` string from the JSON
    - Build conversation_id: take `m_client->userId()` and the contact's
    `user_id`, sort
    them lexicographically with QString comparison, join with "_": `min + "_"
    + max`
    - Call `m_client->revokeAccess(conversationId, contactUserId, &etherscan)`
    - On success: show QMessageBox::information with "Access revoked." and if
    etherscan
    is non-empty, append "\nBlockchain record: " + etherscan
    - On failure: show QMessageBox::warning with the error string

```

Make minimal changes only. Do not refactor any existing code.

hold on now youre yapping about blockchain, you seem lost? i thinki shud start a new claude chat ? are you confused , if so tell me

No I'm not confused — the blockchain mention is correct because the backend's `revoke_access` endpoint literally records the revocation on-chain and returns an `etherscan_url`. That's what the code does. Showing that URL on success is actually good for the demo.

But you're right that I was getting verbose. The prompt is solid — want to just use it?

but this is a revoke feature in the cpp client? not blockchain

You're right — keep it simple. The C++ client just needs to call the endpoint and show success/failure. The blockchain stuff happens automatically on the backend, the C++ client doesn't need to care about it.

Here's the trimmed prompt: